

Resumen Python — Fundamentos a Funciones

Datos y variables · Operadores · Entrada/Salida · Condicionales · Ciclos · Funciones

1. Datos y variables

Python guarda información en **variables**: un nombre que apunta a un valor de un tipo determinado.

- **int** — números enteros (15).
- **float** — números decimales, con **punto**, nunca coma (5.8).
- **str** — texto, entre comillas ("Ana").
- **bool** — True o False.

Asignación: `nombre_variable = valor`. Nombres en **snake_case**, significativos, sin palabras reservadas.

Conversión entre tipos: `int(x)`, `float(x)`, `str(x)`.

```
altura_persona = 175      # int
promedio_notas = 5.8      # float
nombre_estudiante = "Ana" # str
esta_inscrito = True      # bool
```

Palabras reservadas — no usar como nombre de variable

False, None, True, and, or, not, if, elif, else, for, while, def, return, import, break, continue, in

2. Operadores

Aritméticos: + - * / // % **

// es división entera (descarta el resto); % es el resto (módulo); ** es potencia.

```
17 // 5  >> 3
17 % 5   >> 2
2 ** 5   >> 32
```

De comparación (siempre devuelven bool): == != > < >= <=

Lógicos:

- **and** — exige que **ambas** condiciones sean verdaderas.
- **or** — basta con que **una** sea verdadera.
- **not** — invierte el valor lógico.

```
(edad >= 12) and (edad <= 17)
```

3. Entrada y salida

`print(valor1, valor2, ...)` imprime en pantalla, separando con **comas** — nunca concatenar con + y `str()`. **sep=** cambia el separador entre valores; **end=** cambia qué se imprime al final (por defecto, salto de línea).

`input("mensaje")` muestra el mensaje y devuelve **siempre texto** (**str**), aunque la persona escriba números. Para calcular con ese valor hay que convertirlo con `int()` o `float()`.

```
edad = int(input("¿Cuántos años tienes? "))
print("Tienes", edad, "años")
>> Tienes 15 años
```

Convención de este resumen: el output de `print()` se muestra con >> en la línea siguiente.

4. Condicionales

if condición: ejecuta el bloque indentado solo si la condición es **True**. **else:** ejecuta si no se cumplió. La **indentación** (4 espacios) define qué instrucciones pertenecen a cada bloque — no es estética.

elif condición: agrega alternativas intermedias sin anidar — equivale a "si no, y si se cumple esto otro".

```
if nota < 4.0:
    print("Insuficiente")
elif nota < 5.5:
    print("Regular")
else:
    print("Buena")
```

Anidados: un **if/else** dentro de otro. Cada nivel agrega su propia indentación.

```
if es_socia:
    if compra >= 10000:
        print("Doble descuento")
    else:
        print("Descuento socia")
else:
    print("Sin descuento")
```

5. Ciclos

for variable in range(...): repite el bloque una vez por cada valor que entrega **range**. **range(inicio, fin, salto)** — el **fin nunca se incluye**. Con un argumento, empieza en 0; con dos, el salto vale 1.

```
for numero in range(2, 11, 2):
    print(numero)
>> 2
>> 4
>> 6
>> 8
>> 10
```

for anidado: un **for** dentro de otro (tablas, patrones) — el interior completa todo su recorrido por cada paso del exterior.

continue salta el resto de esa iteración y sigue con la siguiente. **break** termina el ciclo completo de inmediato.

```
for numero in range(1, 8):
    if numero == 5:
        break
    if numero % 2 == 0:
        continue
    print(numero)
>> 1
>> 3
```

while condición: repite **mientras** la condición sea **True** — se usa cuando no se sabe de antemano cuántas repeticiones habrá. Hay que actualizar la variable de control dentro del bloque, o el ciclo nunca termina.

```
suma = 0
while suma < 100:
    suma = suma + 10
print("Suma final:", suma)
>> Suma final: 100
```

while True: junto a un **if + break** interno crea un ciclo que se repite hasta que se cumple una condición decidida dentro del propio bloque.

6. Funciones

def nombre_funcion(parametro1, parametro2): define la función; el cuerpo va indentado debajo. **Definir la función todavía no la ejecuta** — solo queda "guardada la receta" hasta que alguien la llama.

```
def calcula_total(precio, cantidad):
    return precio * cantidad
```

Parámetro: variable que la función espera recibir (en la definición). **Argumento:** el valor real que se entrega al llamarla.

return valor entrega el resultado a quien llamó la función y **corta la ejecución ahí mismo**. Sin **return**, la función devuelve **None**.

Patrón clave

resultado = funcion(argumento) — siempre guardar lo que la función devuelve en una variable, para poder usarlo después (imprimirlo, compararlo, sumarlo). Si la función usa **print()** en vez de **return**, esa variable queda en **None**.

```
total = calcula_total(1500, 3)
print("Total:", total)
>> Total: 4500
```

Valores por omisión: **def funcion(parametro=valor):** vuelve ese parámetro opcional al llamar — siempre a la derecha de los parámetros obligatorios.

```
def calcula_total(precio, cantidad=1):
    return precio * cantidad

calcula_total(1500)      # cantidad = 1
calcula_total(1500, 3)  # cantidad = 3
```